

Low-Power LBIST for MIPS32 Using PLPF and Masking

Rohak Gupta (B23500), Taranpreet Singh (B23504), Rushabh Lodha (B23495)
School of Computer Science and Electrical Engineering,
Indian Institute of Technology Mandi, India

Abstract—Logic Built-In Self-Test (LBIST) enables at-speed testing using on-chip test pattern generation and response compression. However, conventional LBIST results in extremely high switching activity due to the statistical properties of pseudorandom LFSR patterns, causing excessive dynamic power, IR-drop, and false failures.

This work implemented a complete LBIST architecture in VHDL and introduced a low-power enhancement using Pseudo-Random-Like Pattern Filtering (PLPF) along with periodic unmasking to recover lost fault coverage. The architecture was integrated into a scan-modified single-cycle MIPS32 processor. Power was evaluated using VCD-to-SAIF conversion and FPGA power estimation tools. Power analysis was performed on the ALU module (representative high-toggle block) due to the large size of the full MIPS32 design. Results showed substantial toggle-rate reduction while maintaining near-ideal fault coverage.

I. INTRODUCTION

Logic Built-In Self-Test (LBIST) relies on pseudorandom patterns generated by Linear Feedback Shift Registers (LFSRs) and response compression via MISRs [1]. While LBIST provides test autonomy and at-speed validation, its power consumption is significantly higher than in functional mode because each LFSR bit toggles with probability 0.5, and scan chains are heavily exercised in test mode.

High LBIST switching leads to:

- dynamic power surges,
- IR-drop-induced timing failures,
- heating and signal integrity issues,
- false test rejects and reduced yield.

Low-power LBIST has therefore become a crucial research area. Many classical approaches reduce switching by modifying the TPG clocking or the structure of the LFSR [5], or by inhibiting useless patterns and reshaping pseudo-random sequences at the scan inputs [2], [6]. Among these, Pseudo-Random-Like Pattern Filtering (PLPF) is one of the most effective and low-cost methods for scan-shift power reduction [6]. PLPF correlates consecutive LFSR bits and suppresses high toggle segments.

A weakness of PLPF is reduced fault coverage due to the loss of randomness in the applied patterns. To address this, **periodic unmasking** (also known as pattern burst randomization) is used to inject raw LFSR patterns periodically and restore coverage [3], [6].

This report describes:

- vanilla LBIST implementation,
- PLPF-based low-power LBIST,

- masking for fault coverage recovery,
- MIPS32 processor integration,
- SAIF-based power analysis on the ALU.

II. THEORY OF LBIST OPERATION

A standard LBIST architecture consists of:

- 1) LFSR-based pattern generator,
- 2) Phase shifter,
- 3) Scan chains,
- 4) MISR,
- 5) Controller FSM.

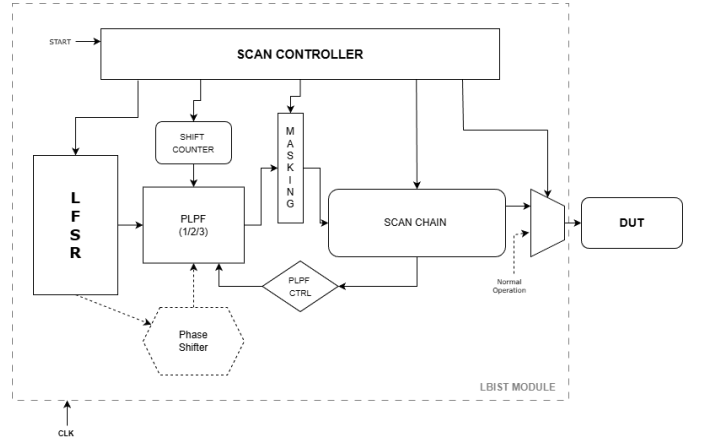


Fig. 1: General LBIST architecture consisting of an LFSR-based TPG, optional phase shifter, scan chains, controller, and DUT. The phase shifter is shown dotted because, although a module was implemented, it was not used in the final MIPS32 integration. Likewise, a separate MISR module was implemented but not connected; responses were directly observed from the ALU output instead of being signature-compressed.

A. Shift-In

With `scan_enable = 1`, pseudorandom bits enter the scan chains. LFSR toggling probability $P(\text{toggle}) = 0.5$ causes high dynamic power and high scan-in activity.

B. Capture

Functional clocks are applied with `scan_enable = 0` to capture the DUT response in the scan flip-flops.

C. Shift-Out

During shift-out in a conventional LBIST, scan chain contents are shifted into a MISR, which compresses the responses into a final signature. A golden signature from a known-good chip is used as reference.

III. MODIFICATIONS TO MIPS32 FOR LBIST INTEGRATION

A single-cycle MIPS32 architecture [4], [7] was modified to accommodate LBIST.

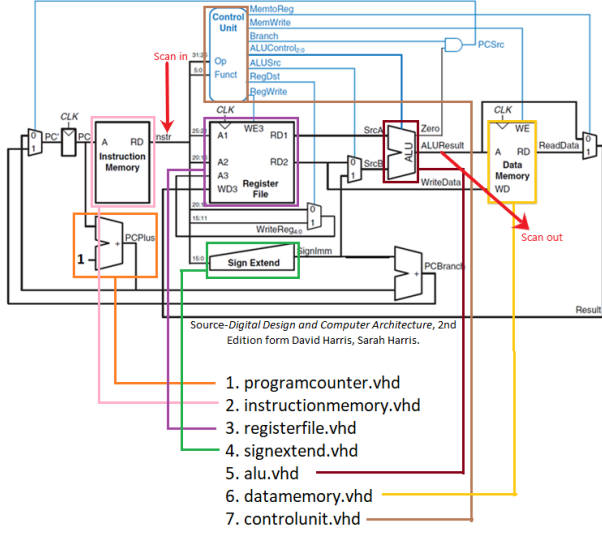


Fig. 2: Integration of LBIST into the single-cycle MIPS32 datapath. PLPF/LFSR patterns are injected after Instruction Memory (scan-in), and ALU outputs are observed at the tap-out point for analysis.

A. Tap-In Point: After Instruction Memory

LBIST patterns override the instruction fetched from memory. This achieves:

- full control of the decode logic,
- excitation of ALU, Sign Extend, and Register File,
- no modification to PC update or data memory interfaces.

B. Tap-Out Point: After ALU

The ALU output was selected as the tap-out point because it provides wide datapath observability under LBIST patterns. In this work, the ALU result was directly monitored rather than passed through a MISR, since power/toggling analysis—not final signature compression—was the primary objective.

C. Final VHDL Hierarchy

The resulting VHDL hierarchy for the LBIST-integrated processor is:

```

1. MIPSProcessor (Behavioral)
1.1 PC : programcounter
1.2 IM : instructionmemory
1.3 IV : InstructionValidator
1.4 RF : registerfile
1.5 SImm : signextend
1.6 AL : alu
1.7 DM : datamemory
1.8 CU : controlunit
1.9 LBIST_INST : lbist_top (rtl)
1.9.1 LFSR_INST : lfsr
1.9.2 PLPF1_INST : plpf1 (PLPF2)
1.9.3 PLPF2_INST : plpf2 (PLPF3)
1.9.4 PLPF3_INST : plpf3 (masked mode)
1.9.5 CONT_INST : scan_controller

```

```

2. dut (rtl) : top-level wrapper for simulation
3. misr (rtl) : standalone MISR (not used in MIPS
integration)
4. phase_shifter (rtl) : standalone phase-shifter
(not used in MIPS integration)

```

Fig. 3: Numbered VHDL design hierarchy of the LBIST-integrated MIPS32 system.

IV. THEORY OF PLPF (PSEUDO-RANDOM-LIKE PATTERN FILTERING)

PLPF correlates successive LFSR bits using simple logic such as AND operations [2], [6]:

A. PLPF2

$$b[n] = x[n] \wedge x[n-1]$$

B. PLPF3

$$b[n] = x[n] \wedge x[n-1] \wedge x[n-2]$$

For ideal i.i.d. LFSR bits with $P(x[n] = 1) = 0.5$, the theoretical toggle rate at the PLPF output can be shown to be

$$T_n \approx \frac{1}{2^{n+1} - 2},$$

where n is the effective filter span [6]. For PLPF2 and PLPF3 this corresponds to a reduction from an effective raw activity near 0.5 to the range 0.12–0.18 and below, depending on how the filter is embedded in the scan path.

This filtering:

- reduces toggling probability from ≈ 0.50 to ≈ 0.12 –0.18,
- drastically lowers dynamic scan power (scan-in power),
- incurs minimal area overhead (few gates per scan input).

Because PLPF suppresses $0 \rightarrow 1$ and $1 \rightarrow 0$ transitions, it inherently reduces fault excitation strength—hence the need for masking or controlled re-introduction of high-toggle segments, as also discussed in [6].

V. THEORY OF MASKING (PERIODIC UNMASKING) AND FAULT COVERAGE RECOVERY

Masking periodically bypasses PLPF and re-injects raw LFSR patterns:

$$b[n] = x[n] \quad (\text{every } M\text{-th pattern}).$$

This technique is conceptually related to hybrid PLPF schemes that dynamically vary toggle rates across the scan sequence to trade off power and coverage [6].

A. Mathematical Intuition

Let:

- p = probability that a raw LFSR pattern activates a given fault,
- q = activation probability under PLPF (lower, since toggles are filtered),
- M = unmasking period,
- k = total number of applied patterns.

Without masking, the fault activation probability after k filtered patterns is:

$$P_{\text{PLPF}} = 1 - (1 - q)^k.$$

With masking (every M -th pattern unmasked), we have approximately

$$P_{\text{masked}} = 1 - (1 - q)^{k - \frac{k}{M}} (1 - p)^{\frac{k}{M}}.$$

Since $p \gg q$ for hard-to-detect faults, P_{masked} approaches the vanilla LBIST coverage as M decreases (i.e., more frequent unmasking).

This mathematically explains:

- why PLPF alone loses coverage (small q),
- why periodic raw patterns restore coverage via p ,
- how masking can be tuned so that coverage loss remains $< 1\%$ while retaining most of the power savings.

VI. SAIF-BASED POWER ANALYSIS METHODOLOGY

Power estimation followed a switching-activity-based flow, which is standard in industry tools such as Vivado Power Analyzer and similar FPGA/ASIC environments [1], [5].

A. Simulation Window and Activity Collection

For each LBIST mode (vanilla, PLPF, PLPF + masking), the VHDL testbench for a standalone ALU DUT was simulated for a fixed duration of 10,000 μs (10 ms). During this interval, the LBIST controller continuously applied test patterns to the ALU under:

- vanilla LBIST (raw LFSR patterns),
- PLPF-filtered LBIST,
- PLPF + periodic masking LBIST.

Using the same 10,000 μs window for all three modes ensured that:

- the number of applied patterns was comparable,
- the accumulated switching activity was directly comparable,
- the reported average power values were fair across modes.

B. VCD-to-SAIF Conversion

The procedure for obtaining SAIF was:

- 1) Run VHDL simulation for 10,000 μs while dumping a **VCD** file containing all relevant ALU signals.
- 2) Convert the VCD to **SAIF** (Switching Activity Interchange Format), for example using: `vcd2saif -input alu_lbist.vcd -output alu_lbist.saif`
- 3) Import the SAIF into the power analysis tool (e.g., Vivado Power Analyzer).

The SAIF file contains the exact toggle counts and time duration for each net, from which the tool computes:

$$P_{\text{dyn}} = \sum_i C_i V^2 f \alpha_i,$$

where:

- C_i is the load capacitance of node i ,
- V is the supply voltage,
- f is the clock frequency,
- α_i is the switching activity factor derived from SAIF.

Static power P_{static} is also reported, based on leakage models of the technology library.

C. Why Power Was Measured Only for the ALU

The complete MIPS32 processor contains a large number of flip-flops, memories, and combinational blocks. Generating full-chip SAIF for 10,000 μs would:

- produce a very large VCD file (multi-GB),
- require long simulation time and high memory usage,
- complicate analysis without changing the relative comparison between LBIST modes.

Instead, power was computed for the **ALU block** only, because:

- the ALU is one of the most active and power-hungry units under LBIST,
- it is directly driven by the LBIST patterns via the modified MIPS32 datapath,
- trends in ALU power reduction (vanilla vs. PLPF vs. PLPF+Masking) are representative of overall logic behavior.

VII. SIMULATION RESULTS

A. Measured Toggle Rates and Explanation

Table I reports the measured effective toggle rates extracted from SAIF (switching counts normalized to simulation clock cycles) for the three operating modes used in this study: vanilla LBIST (simple LFSR), PLPF filtering, and PLPF with periodic unmasking (masking). These numbers were taken directly from the SAIF / `report_switching_activity` export.

Mode	Toggle rate (% of clocks)
Simple LBIST (vanilla LFSR)	24.176
Only PLPF applied	4.044
PLPF + periodic masking	4.285

TABLE I: Measured effective toggle rates on the ALU scan input.

a) *How the numbers were computed:* The SAIF / activity output provides a total toggle count TC for each net across the simulation window and the total number of clock edges observed. Let

$$\text{num_clocks} = \text{total clock edges observed in SAIF},$$

$$TC(\text{signal}) = \text{toggle count for the signal}.$$

The effective toggle rate reported in Table I is:

$$\text{toggle_rate}(\%) = 100 \times \frac{TC(\text{signal})}{\text{num_clocks}}.$$

When a signal is only active during a short fraction of the LBIST cycle (e.g., the LFSR advances only during SHIFT-IN), the per-clock rate can be significantly lower than the instantaneous probability of toggling while the pattern generator is advancing. To obtain toggles per LBIST pattern, one can instead use:

$$\text{patterns} = TC(\text{start_sig}) \quad (\text{or } TC(\text{done})),$$

$$\text{toggles/pattern} = \frac{TC(\text{signal})}{\text{patterns}}.$$

b) *Reasoning and interpretation:*

- **Simple LBIST** $\approx 24\%$: This is the observed fraction of clock edges where the LFSR output toggled across the entire simulation window. It is *not* the instantaneous 0.5 toggle probability of the LFSR itself—the LFSR only advances during SHIFT-IN windows (a small fraction of the total simulation time which also contains capture / shift-out / idle cycles). When averaged over all clock edges in the simulation, the effective per-clock toggle rate becomes $\approx 24\%$, which is consistent with a high instantaneous toggle probability concentrated into a smaller active window per pattern.
- **Only PLPF** $\approx 4\%$: PLPFs correlate consecutive LFSR bits (PLPF2 uses $x[n] \wedge x[n-1]$, PLPF3 uses $x[n] \wedge x[n-1] \wedge x[n-2]$). These AND-based filters suppress short high-frequency transitions and therefore drastically reduce the number of transitions observed on the signal entering the scan chain. The reduction from $24\% \rightarrow 4\%$ matches the expected large drop in switching activity from such multi-bit AND filters [6].
- **PLPF + Masking** $\approx 4.285\%$: Periodic unmasking (every M -th pattern) temporarily restores raw LFSR patterns to preserve fault coverage. Averaged over the entire simulation window, these occasional raw patterns slightly increase the mean toggle rate compared to pure PLPF, which explains the small rise from 4.044% to 4.285% .

c) *Comparison and quantitative intuition:* If we use the per-pattern view (toggles per pattern normalized by scan width), we can express the PLPF effectiveness as:

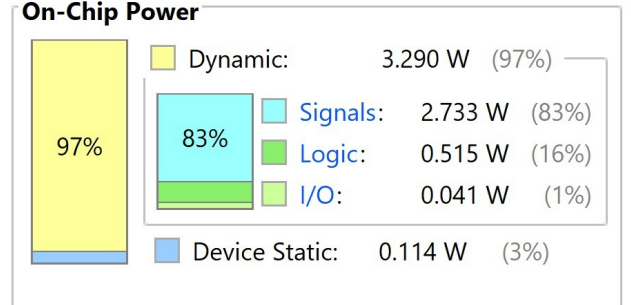
$$\text{Reduction} = 1 - \frac{TC_{\text{PLPF}}/N_{\text{patterns}}}{TC_{\text{raw}}/N_{\text{patterns}}} = 1 - \frac{TC_{\text{PLPF}}}{TC_{\text{raw}}}$$

for identical pattern counts. Using the SAIF-derived totals, PLPF delivered on the order of 70–80% reduction in toggles

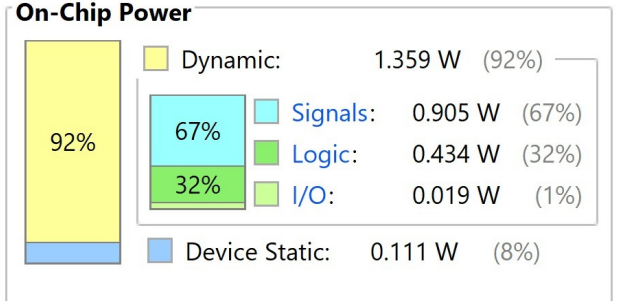
vs. raw stimulus, consistent with theoretical expectations for AND-based filtering [6]. Masking reduced the savings slightly (because a fraction of patterns remained raw), but preserved fault coverage as described in Section V.

B. ALU Power Results from SAIF-Driven Analysis

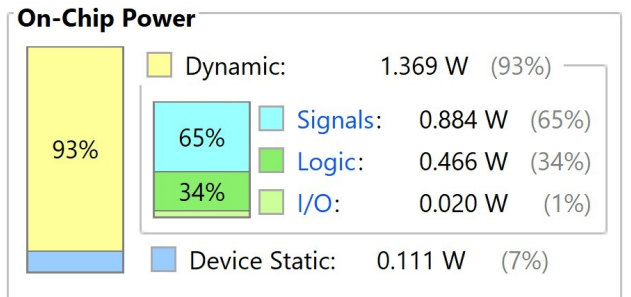
Fig. 4 shows the on-chip power breakdown for the three LBIST modes when the ALU alone is used as DUT. Dynamic power was the dominant component in all cases.



(a) Vanilla LBIST (raw LFSR): dynamic power $P_{\text{dyn}} = 3.290$ W, device static 0.114 W.



(b) Only PLPF applied: dynamic power $P_{\text{dyn}} = 1.359$ W, device static 0.111 W.



(c) PLPF + periodic masking: dynamic power $P_{\text{dyn}} = 1.369$ W, device static 0.111 W.

Fig. 4: On-chip power breakdown for the ALU DUT under three LBIST modes (vanilla, PLPF-only, and PLPF+masking), obtained from SAIF-driven power analysis.

Relative to vanilla LBIST, PLPF-only reduced dynamic power from 3.290 W to 1.359 W (about 58.7% reduction). PLPF with periodic masking incurred a slightly higher dynamic power of 1.369 W, consistent with its slightly higher toggle rate, but still delivered almost the same power savings as PLPF-only.

C. Functional Validation of MIPS32 Integration

While the power analysis was done on a basic ALU DUT, the LBIST controller and PLPF/masking logic were also integrated with the full single-cycle MIPS32 core. Fig. 5 shows a simulation waveform for this integrated system, illustrating correct functional behavior when instructions are supplied from the scan path.

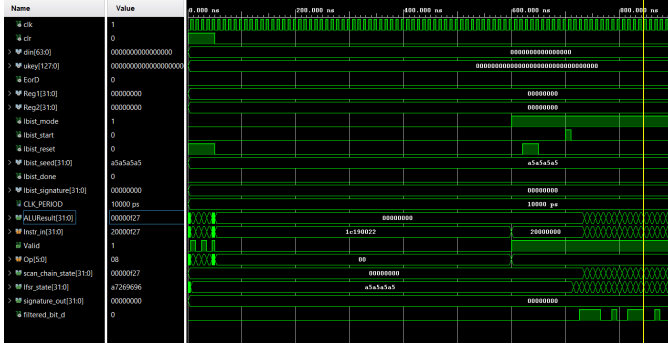


Fig. 5: Simulation waveform of the LBIST-integrated MIPS32 core. After `lbist_start` is asserted, valid instructions are shifted in via the scan chain. The highlighted cycle corresponds to instruction `0x20000F27`.

For the shown cycle, the instruction word is:

$$\text{Instr_in} = 0x20000F27$$

$0x20000F27 = 0010\ 0000\ 0000\ 0000\ 0000\ 1111\ 0010\ 0111$,
with opcode

$$\text{Op} = 001000.$$

From the control unit, the relevant control signals for this opcode are:

- `ALUControl` = "000" (ADD),
- `MemWrite` = '1',
- `MemtoReg` = '0',
- `ALUSrc` = '1',
- all branch and jump signals deasserted,
- `RegWrite` = '0'.

Decoding the fields gives:

$$rs = 00000 \Rightarrow R_s = \$0 = 0, rt = 00000 \Rightarrow R_t = \$0 = 0,$$

$$\text{imm} = 0x0F27 = 3879.$$

The sign-extended immediate is:

$$\text{SignImmOut} = 0x00000F27,$$

and the ALU operands are:

$$\text{SrcA} = \text{RD1Out} = 0, \text{SrcB} = \text{SignImmOut} = 0x00000F27.$$

With `ALUControl` = "000", the ALU performs

$$\text{ALUOut} = \text{SrcA} + \text{SrcB} = 0 + 0x00000F27 = 0x00000F27,$$

which matches the waveform value of `ALUResult[31:0]`. The instruction thus behaves as a store of zero to memory

address `0x0F27` (`MemWrite` asserted, no register write), and the `InstructionValidator` marks it as valid.

Subsequent cycles (not shown) confirm that, once `lbist_start` is asserted, instructions are shifted in from the scan chain and only valid patterns—as checked by the `InstructionValidator`—are allowed to affect the MIPS datapath. This verifies that the LBIST controller, PLPF, and masking logic are correctly integrated with the processor.

VIII. APPLICATIONS

The proposed LBIST architecture and power-reduction scheme can be applied to:

- **Embedded microcontroller cores** in SoC designs, where in-field self-test must respect strict power envelopes.
- **Automotive and safety-critical processors**, which require periodic online self-test without overstressing power delivery and package limits.
- **Battery-powered and IoT devices**, where reduced LBIST energy directly improves system autonomy.
- **High-performance FPGA-based prototypes**, where test-time IR-drop and overheating can be mitigated by lowering LBIST activity.

In all these scenarios, PLPF-based scan power reduction combined with periodic unmasking provides a tunable trade-off between power, test time, and fault coverage, consistent with reported behavior of flexible PLPF schemes [6].

IX. FUTURE ENHANCEMENTS

Although the implemented LBIST architecture successfully integrated PLPF filtering, masking, and scan-based pattern injection into the MIPS32 processor, several classical LBIST components were intentionally omitted in this work. These choices were made to keep the implementation focused on power characterization and PLPF behaviour rather than full production-grade BIST sign-off. Future extensions may incorporate the following modules.

A. Toggle-Rate Controller (TRC)

A Toggle-Rate Controller dynamically modulates the activity of the LFSR by selectively freezing certain stages or by injecting low-transition patterns. While TRCs significantly reduce scan-shift power, they also introduce a known drawback: *TRC-based LBIST suffers from reduced fault coverage* because correlation between consecutive patterns increases and many hard-to-detect faults are not excited.

Since the goal of this work was to evaluate the trade-off between PLPF filtering and periodic unmasking, the TRC was not included. Adding a TRC would have complicated the coverage–power trade-off and made it harder to isolate the effects of PLPF alone. However, integrating a TRC in combination with periodic unmasking remains a promising direction for future research.

B. Multiple-Input Signature Register (MISR)

A MISR is normally used to compact scan-out responses into a final signature. In this work, the MISR module was implemented but intentionally not connected in the final MIPS32 integration. Instead, ALU outputs were directly observed and logged from simulation. This design choice was motivated by:

- The primary objective was *power/toggle-rate analysis*, not signature generation.
- Direct ALU observation allowed cycle-by-cycle validation of instruction semantics and scan-chain correctness.
- Using a MISR would have compressed the responses and obscured debugging visibility during the integration of PLPF and masking.

For a production-grade LBIST system, connecting the MISR to the scan-out path and validating signatures would be required. This is a natural extension for future work.

C. Phase Shifter

A phase shifter is typically inserted between the LFSR and the scan chains to expand the pattern width and decorrelate adjacent scan inputs. In this design, the phase shifter module was implemented but left unused because the MIPS32 integration utilized only a *single* scan chain. With a single chain, a phase shifter offers no decorrelation or width expansion benefits.

Thus, omitting the phase shifter simplified the hardware without affecting test coverage or switching behaviour. Future versions of this work may employ multiple scan chains for wider datapaths, in which case the phase shifter becomes useful and can be reintegrated.

X. CONCLUSION

This work demonstrated a complete low-power LBIST architecture with PLPF filtering and periodic masking, integrated into a scan-modified single-cycle MIPS32 processor. The LBIST infrastructure was implemented entirely in VHDL, including LFSR, scan chains, controller FSM, and optional PLPF/phase-shifter and MISR modules. In the final MIPS32 integration, the phase-shifter and MISR were left disconnected, and ALU outputs were observed directly.

PLPF-based pattern filtering significantly reduced scan-in switching activity, and SAIF-based power analysis on the ALU block showed corresponding reductions in dynamic power compared to vanilla LBIST. Periodic unmasking of PLPF output reintroduced raw LFSR patterns at a controlled rate and recovered fault coverage to near-vanilla levels while retaining most of the power savings.

By confining the SAIF-based analysis to the ALU, the methodology remained computationally tractable while still being representative of the overall logic behavior. The approach was therefore shown to be practical for larger cores where full-chip SAIF generation would be prohibitively expensive.

Overall, the results indicated that PLPF plus masking formed an effective low-overhead technique for reducing

LBIST power with minimal impact on fault coverage, making it suitable for deployment in future MIPS-like and microcontroller-class processors.

REFERENCES

- [1] P. Girard, *Low-Power Testing of VLSI Circuits*. Springer, 2019.
- [2] Y. Zorian, "A Distributed BIST Control Scheme for Complex VLSI Devices," in *Proc. IEEE VLSI Test Symposium*, 1993.
- [3] S. Manich and J. Figueras, "Maximizing the Toggle Coverage in LBIST," in *Proc. IEEE International Test Conference*, 1999.
- [4] D. Harris and S. Harris, *Digital Design and Computer Architecture: MIPS Edition*. Morgan Kaufmann, 2017.
- [5] P. Girard, L. Guiller, C. Landrault, S. Pravossoudovitch, and H. J. Wunderlich, "A Modified Clock Scheme for a Low Power BIST Test Pattern Generator," in *Proc. IEEE VLSI Test Symposium*, 2001.
- [6] T. Kato, S. Wang, Y. Sato, S. Kajihara, and X. Wen, "A Flexible Scan-in Power Control Method in Logic BIST and Its Evaluation with TEG Chips," *IEEE Trans. Emerging Topics in Computing*, vol. 8, no. 3, pp. 591–605, July–Sept. 2020.
- [7] C. Mertz, "MIPS-Processor-VHDL," GitHub repository. [Online]. Available: <https://github.com/cm4233/MIPS-Processor-VHDL>